

Animal Classification Using Transfer Learning with MobileNetV2

Author: Vijay Bedage

Affiliation: MCA Student, KLE Technological University, Hubli

Date: July 2026

Table of Contents

1. Introduction
2. Literature Survey
3. Objectives
4. Problem Statement
5. Methodology
6. Applications
7. Results and Discussion
8. Conclusion
9. References

1. Introduction

Image classification is a fundamental computer vision task that assigns a label to an input image from predefined categories. With growing volumes of wildlife imagery from camera traps and ecological surveys, the demand for automated animal species identification has increased significantly. Manual identification is time-consuming, expensive, and error-prone.

Deep learning with Convolutional Neural Networks (CNNs) has revolutionized image classification by learning hierarchical features from raw pixels. However, training deep models from scratch requires millions of labelled images and heavy computation—impractical for wildlife datasets that are typically small and imbalanced.

Transfer learning addresses this by reusing models pre-trained on large-scale datasets like ImageNet (1.2 million images, 1,000 classes). By fine-tuning pre-trained features on smaller domain-specific datasets, transfer learning achieves high accuracy with limited data.

This project presents an animal classification system using **MobileNetV2** with transfer learning to classify images into **15 animal species**: Bear, Bird, Cat, Cow, Deer, Dog, Dolphin, Elephant, Giraffe, Horse, Kangaroo, Lion, Panda, Tiger, and Zebra. It features a two-phase training pipeline, data augmentation, and a Streamlit web application for real-time predictions.

2. Literature Survey

2.1 MobileNetV2: Inverted Residuals and Linear Bottlenecks

Sandler et al. (2018) introduced MobileNetV2, a lightweight CNN designed for mobile and resource-constrained environments. It features **inverted residual blocks** (shortcut connections between thin bottleneck layers) and **linear bottlenecks** (no ReLU in low-dimensional layers to prevent information loss). Using depthwise separable convolutions, it achieves 72.0% top-1 accuracy on ImageNet with only 3.4M parameters, making it ideal for transfer learning [1].

2.2 Transfer Learning for Animal Species Classification

Adegun et al. (2023) studied transfer learning with GoogLeNet, ResNet50, and VGG19 for camera-trap animal classification. Transfer learning consistently outperformed training from scratch, achieving 83–87% accuracy. They highlighted data augmentation as essential for combating data scarcity and environmental complexity in wildlife images [2].

2.3 Animal Classification Using MobileNetV2

Pratama et al. (2023) applied MobileNetV2 with transfer learning on the Animals-10 dataset, achieving >95% accuracy. Their pipeline—preprocessing (224×224 resize), augmentation (flip, rotate, scale), Adam optimizer, and categorical cross-entropy—directly informs the methodology of this project [3].

2.4 DenseNet-Based Multi-Species Recognition

Rahman et al. (2024) used DenseNet-161 for 90-class animal recognition with limited data, achieving 85–89% accuracy. While DenseNet offers higher accuracy for large class counts, MobileNetV2 is preferable for real-time deployment due to its significantly lower computational footprint [4].

Study	Architecture	Accuracy	Key Finding
Sandler et al. (2018)	MobileNetV2	72.0%	Inverted residuals, linear bottlenecks
Adegun et al. (2023)	GoogLeNet, ResNet50	83–87%	Transfer learning for wildlife
Pratama et al. (2023)	MobileNetV2	>95%	Fine-tuning for animal classification
Rahman et al. (2024)	DenseNet-161	85–89%	Dense connectivity with limited data

3. Objectives

1. Develop an image classification system to identify 15 animal species using MobileNetV2 transfer learning.
2. Implement a two-phase training strategy—frozen base feature extraction followed by fine-tuning.
3. Apply data augmentation to improve generalization on a small dataset (~1,944 images).
4. Deploy the model via an interactive Streamlit web application for real-time predictions.
5. Evaluate using accuracy, precision, recall, F1-score, and confusion matrix analysis.

4. Problem Statement

How can we build a lightweight, efficient, and accurate animal image classification system that achieves >90% accuracy across 15 species while remaining suitable for real-time deployment?

Key challenges:

- **Limited data:** ~1,944 images across 15 classes (~130 per class)—insufficient for training from scratch.
- **Inter-class similarity:** Tiger vs. Lion, Cat vs. Dog, Horse vs. Deer present classification difficulties.
- **Computational constraints:** Model must be compact (~14 MB) with fast inference (<100ms).
- **Generalization:** Must handle unseen images with varying lighting, angles, and backgrounds.

5. Methodology

5.1 Project Configuration

All hyperparameters and settings are centralized in `config.py`:

```
# src/config.py - Central Configuration
import os

# Paths
PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
DATASET_PATH = os.path.join(PROJECT_ROOT, 'Animal Classification', 'dataset')
MODEL_PATH = os.path.join(PROJECT_ROOT, 'models', 'animal_classifier.h5')

# Model Hyperparameters
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 30
LEARNING_RATE = 0.001
FINE_TUNE_EPOCHS = 10
FINE_TUNE_LR = 1e-5
FINE_TUNE_LAYERS = 30
SEED = 42

# 15 Animal Classes
CLASSES = [
    'Bear', 'Bird', 'Cat', 'Cow', 'Deer', 'Dog', 'Dolphin',
    'Elephant', 'Giraffe', 'Horse', 'Kangaroo', 'Lion',
    'Panda', 'Tiger', 'Zebra'
]
NUM_CLASSES = len(CLASSES)
```

5.2 Dataset and Data Augmentation

The dataset contains **~1,944 images** across 15 classes, automatically split into 80% training (~1,555) and 20% validation (~389). Real-time augmentation is applied only to training data:

```
# src/train.py - Data Generators
from tensorflow.keras.preprocessing.image import ImageDataGenerator

def build_generators():
    train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=20,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        validation_split=0.2
    )
    val_datagen = ImageDataGenerator(
        rescale=1./255, validation_split=0.2
    )
    train_gen = train_datagen.flow_from_directory(
        DATASET_PATH, target_size=IMG_SIZE, batch_size=BATCH_SIZE,
        class_mode='categorical', subset='training',
        shuffle=True, seed=SEED, classes=CLASSES
    )
    val_gen = val_datagen.flow_from_directory(
        DATASET_PATH, target_size=IMG_SIZE, batch_size=BATCH_SIZE,
        class_mode='categorical', subset='validation',
        shuffle=False, seed=SEED, classes=CLASSES
    )
    return train_gen, val_gen
```

5.3 Model Architecture

MobileNetV2 (pretrained on ImageNet) is used as the feature extractor with a custom classification head:

Input (224×224×3) → **MobileNetV2** (frozen) → GlobalAvgPool → BatchNorm
 → Dense(256, ReLU) → Dropout(0.5) → Dense(128, ReLU) → Dropout(0.3) → **Dense(15, Softmax)**

```
# src/train.py - Model Building
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import (Dense, GlobalAveragePooling2D,
                                     Dropout, BatchNormalization)
from tensorflow.keras.models import Model

def build_model(num_classes=NUM_CLASSES):
    base_model = MobileNetV2(
        input_shape=(*IMG_SIZE, 3),
        include_top=False, weights='imagenet'
    )
    base_model.trainable = False # Freeze base (Phase 1)

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = BatchNormalization()(x)
    x = Dense(256, activation='relu')(x)
    x = Dropout(0.5)(x)
    x = Dense(128, activation='relu')(x)
    x = Dropout(0.3)(x)
    outputs = Dense(num_classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=outputs)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=LEARNING_RATE),
        loss='categorical_crossentropy', metrics=['accuracy']
    )
    return model, base_model
```

5.4 Two-Phase Training

Phase 1 trains only the classification head. Phase 2 unfreezes the top 30 MobileNetV2 layers for fine-tuning:

```
# src/train.py - Fine-Tuning (Phase 2)
def fine_tune_model(model, base_model):
    base_model.trainable = True
    for layer in base_model.layers[:-FINE_TUNE_LAYERS]:
        layer.trainable = False # Keep early layers frozen
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=FINE_TUNE_LR),
        loss='categorical_crossentropy', metrics=['accuracy']
    )
```

```

# src/train.py - Main Training Loop
if __name__ == '__main__':
    set_seed(SEED)
    train_gen, val_gen = build_generators()
    model, base_model = build_model()

    # Phase 1: Train classification head (base frozen)
    history1 = model.fit(
        train_gen, epochs=EPOCHS, validation_data=val_gen,
        callbacks=[
            EarlyStopping(monitor='val_accuracy', patience=7,
                           restore_best_weights=True),
            ModelCheckpoint(MODEL_PATH, monitor='val_accuracy',
                            save_best_only=True),
            ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                              patience=3, min_lr=1e-7)
        ]
    )

    # Phase 2: Fine-tune top 30 layers
    fine_tune_model(model, base_model)
    history2 = model.fit(
        train_gen, epochs=FINE_TUNE_EPOCHS, validation_data=val_gen,
        callbacks=[EarlyStopping(...), ModelCheckpoint(...), ReduceLROnPlateau(...)]
    )

```

5.5 Prediction & Web Application

```

# src/predict.py - Command-Line Inference
def predict_image(image_path, model_path=MODEL_PATH):
    model = tf.keras.models.load_model(model_path)
    img = Image.open(image_path).convert('RGB').resize(IMG_SIZE)
    arr = np.array(img) / 255.0
    arr = np.expand_dims(arr, axis=0)
    probs = model.predict(arr, verbose=0)[0]
    predicted_class = CLASSES[np.argmax(probs)]
    confidence = np.max(probs)
    print(f"Predicted: {predicted_class} | Confidence: {confidence*100:.2f}%")

```

```
# app.py - Streamlit Web App (key excerpt)
import streamlit as st
import plotly.graph_objects as go

@st.cache_resource
def load_model():
    return tf.keras.models.load_model(MODEL_PATH)

def predict(model, img):
    img_resized = img.resize(IMG_SIZE)
    arr = np.array(img_resized) / 255.0
    arr = np.expand_dims(arr, axis=0)
    probs = model.predict(arr, verbose=0)[0]
    top5_idx = np.argsort(probs)[:5][::-1]
    return probs, top5_idx

uploaded_file = st.file_uploader("Upload an animal image...",
                                  type=['jpg', 'jpeg', 'png'])

if uploaded_file:
    img = Image.open(uploaded_file).convert('RGB')
    model = load_model()
    probs, top5_idx = predict(model, img)
    predicted_class = CLASSES[top5_idx[0]]
    st.success(f"Predicted: {predicted_class} - {probs[top5_idx[0]]*100:.2f}%")
```

5.6 Evaluation — Confusion Matrix

```
# src/train.py - Confusion Matrix
from sklearn.metrics import classification_report, confusion_matrix

def plot_confusion_matrix(model, val_gen):
    val_gen.reset()
    preds = model.predict(val_gen, verbose=0)
    y_pred = np.argmax(preds, axis=1)
    y_true = val_gen.classes
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=CLASSES, yticklabels=CLASSES)
    plt.savefig(os.path.join(RESULTS_PATH, 'confusion_matrix.png'))
    report = classification_report(y_true, y_pred, target_names=CLASSES)
    with open('classification_report.txt', 'w') as f:
        f.write(report)
```

6. Applications

1. **Wildlife Conservation:** Automated species identification from camera-trap images for population monitoring and endangered species tracking.
2. **Biodiversity Assessment:** Rapid species cataloguing for environmental impact assessments.
3. **Smart Agriculture:** Differentiating livestock from predators for automated alert systems.
4. **Education:** Interactive learning tool for students via the web application with fun facts.
5. **Zoo Management:** Automated tracking and behaviour monitoring in zoological facilities.

- 6. **Mobile Deployment:** Lightweight model (~14 MB, <100ms inference) suitable for smartphones, drones, and IoT devices.
- 7. **Citizen Science:** Public wildlife surveys via mobile app integration.

7. Results and Discussion

7.1 Training Performance

Metric	Phase 1 (Frozen Base)	Phase 2 (Fine-Tuned)
Best Validation Accuracy	89.66%	~88.11%
Final Training Accuracy	89.6%	84.8%
Best Validation Loss	0.410	0.387

7.2 Training Progression

- **Phase 1 (15 epochs):** Accuracy rose from 42.9% → 89.6%. Learning rate auto-reduced (0.001 → 0.0005 → 0.00025) via ReduceLROnPlateau.
- **Phase 2 (6 epochs):** Initial accuracy dip after unfreezing layers (expected), followed by recovery. LR further reduced to 5e-6.

7.3 Per-Class Analysis

- **High-confidence:** Zebra, Giraffe, Elephant, Panda—unique visual features yield near-perfect classification.
- **Challenging pairs:** Cat/Dog, Tiger/Lion, Horse/Deer—morphological similarity causes occasional misclassifications, consistent with literature [2][4].

7.4 Model Characteristics

Property	Value
Model Size	~14 MB
Inference Time	<100 ms/image
Input Resolution	224 × 224 × 3
Output	15-class softmax

The ~90% accuracy with only ~130 images/class validates MobileNetV2 transfer learning for small-dataset scenarios, comparable to Pratama et al.'s >95% on larger per-class data [3]. Data augmentation played a critical role in preventing overfitting, as confirmed by the relatively small gap between training and validation accuracy.

8. Conclusion

This project successfully developed an animal classification system using **MobileNetV2 transfer learning** for **15 species** with **~90% validation accuracy**. Key outcomes:

- **Two-phase training** effectively leverages pretrained features with limited data (~1,944 images).
- **MobileNetV2** provides optimal accuracy-efficiency trade-off (~14 MB, <100ms inference).
- **Data augmentation** (rotation, shifting, zooming, flipping) prevents overfitting and improves generalization.
- **Streamlit deployment** enables accessible real-time classification with interactive visualizations.

Future Scope: Expanding the dataset, exploring EfficientNet/Vision Transformers, implementing Grad-CAM explainability, TensorFlow Lite mobile deployment, and fine-grained subspecies classification.

9. References

- [1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE CVPR*, 2018, pp. 4510–4520.
- [2] A. A. Adegun, S. Viriri, and R. O. Ogundokun, "Deep Learning Approach for Animal Species Classification Using Transfer Learning with CNNs," *Int. J. Intelligent Systems and Applications in Engineering*, vol. 11, no. 4, pp. 356–365, 2023.
- [3] R. Pratama, A. Mulyana, and R. Adiputra, "Animal Image Classification Using MobileNet-V2 Transfer Learning," *Indonesian Computer Journal*, vol. 8, no. 2, pp. 112–121, 2023.
- [4] M. M. Rahman, S. Ahmed, and K. Deb, "Multi-Species Animal Recognition Using DenseNet-Based Deep Transfer Learning with Limited Training Data," *TechRxiv Preprint*, 2024.

Tech Stack: Python 3.10+ · TensorFlow/Keras · MobileNetV2 · scikit-learn · Streamlit · Plotly · Matplotlib/Seaborn

Repository: github.com/Vijaybedage/ML-project